

SimPy, To Simulate With Python

PRIYA RAVINDRAN

Real-life activities are a constant interaction between components and their environment. A discrete system is no different, constantly taking part in process after process, all the while involved with other systems and its surroundings. Each of these interactions is termed an event, which forms the basis of how these systems work.

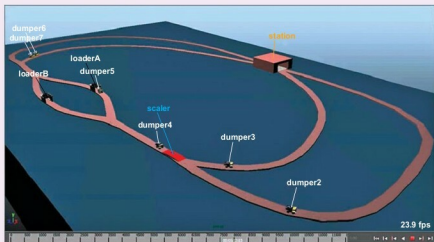
This article looks at SimPy, a tool that would help you simulate these processes and events using Python generators. SimPy is a process based discrete event simulation framework. With SimPy, you can implement asynchronous networks or multi-agent systems involving both real and simulated communication, and its processes can model active components like customers, vehicles or agents.

Little things thoughtfully put together

"If you break SimPy down, it is just an asynchronous event dispatcher," say the developers of this simulation tool. Priority, simulation time, event identifier, call-backs and return values are the defining factors of SimPy's functionality, apart from being the differentiators and the deciders.

Every event needs to be handled uniquely. If you have a digital board display and you need to show certain phrases in a given order and for a given time, start by setting the environment for this. Here, this could be the necessary criterion for each display or setting up of the setup itself. Each of these is a process that would be triggered by an event like the closing of a switch.

SimPy makes sure that each



Discrete event simulation with SimPy and Maya, a 3D computer graphics software (Image courtesy: www.youtube.com)

One-shot look

Under the microscope: SimPy

What it does: Discrete event simulation

Stand-out feature: Easy to use

How to use it: Code using Python

Released under: MIT licence

Operating system: Cross-platform

Latest release: Version 3.0.9

Can be found on: Bitbucket.org

Homepage: <https://simpy.readthedocs.org>

process happens smoothly, suspending it when an event occurs and resuming it once triggered. A single event can trigger multiple processes, and these happen in the order in which these yield the event.

It need not always go as planned. Sometimes, the process gets into a never-ending loop. To avoid such incidents, SimPy offers Timeout feature that is triggered after a certain amount of time and puts the process to sleep. There is also the case of one process being influenced by another.

Take the case of an electric car. The car cannot start until its battery

is charged. But you might choose to begin driving the car while the battery is still not fully-charged. Calling these in terms of processes brings two of the most obvious features, Wait and Interrupt.

When there is demand. Taking the case of the electric car a little further, let us look at a scenario of charging it—a charging station equipped with two charging units and four cars waiting in queue. To model such a scenario, SimPy introduces the concept of shared resources in first-in-first-out (FIFO) manner.

Shared resources come in handy while modelling limited-capacity congestion points like servers, checkout counters and tunnels. While the first process gets the resource first, it can be configured to release the resource when the job is done, or the release could be automated. The resource is then handed over to the process waiting in line.

Talking about resources, you can choose the resource accord-